

- GAME DESIGN DOKUMENT -

BLACKOUT PROTOCOL

MARIA HUHNSTOCK

Inhaltsverzeichnis

1. Pitch..... 1
2. Design Pillars..... 1
3. Core Loop..... 1
4. Mechaniken..... 2
    4.1 Energiesystem..... 2
    4.2 Gebäude..... 2
    4.3 Licht & Sichtbarkeit..... 2
    4.4 Gegner (2 Typen)..... 3
    4.5 Wellen..... 3
    4.6 Schrott & Wiederaufbau..... 4
    4.7 Rang & Auswertung..... 4
    4.8 Playtesting & Balancing..... 4
    4.9 Audio..... 4
5. UI / UX..... 5
6. Artstyle..... 5
7. Technische Struktur..... 6
8. Scope..... 8
9. Mögliche Erweiterungen..... 8

Blackout Protocol - Game Design Dokument

GENRE: Top-Down-Strategie / Tower Defense **ENGINE:** Unity 6.6 (URP), C#

1. Pitch

Ein Außenposten bei Nacht, ein Generator mit genau 10 Energieeinheiten und drei Gegnerwellen, die aus der Dunkelheit heranrücken. Du baust nichts – alles steht schon. Deine Aufgabe ist es, Sekunde für Sekunde zu entscheiden, was Strom bekommt: die Lampen, die Gegner sichtbar machen, das Geschütz, das auf sie schießt, oder die Drohne, die repariert, was sie beschädigen. Für alles gleichzeitig reicht es nie.

2. Design Pillars

- **Energie ist die Währung** - jede Entscheidung dreht sich um dieselben 10 Einheiten. Lampen, Geschütz und Drohne brauchen zusammen 11, irgendetwas bleibt also immer aus.
- **Licht bestimmt das Schlachtfeld** - das Geschütz trifft nur Gegner, die im Licht stehen. Eine Lampe abzuschalten spart Energie, öffnet aber einen toten Winkel.
- **Lesbar im Dunkeln** - klare Silhouetten, farblich unterscheidbare Gebäude und eine ständig sichtbare Energieanzeige machen die Lage auf einen Blick verständlich, auch wenn der Großteil der Karte schwarz ist.
- **Aufholen statt neu starten** - eine verlorene Lampe ist ein Rückschlag, nicht das Ende. Ruinen lassen sich zwischen den Wellen wieder aufbauen.

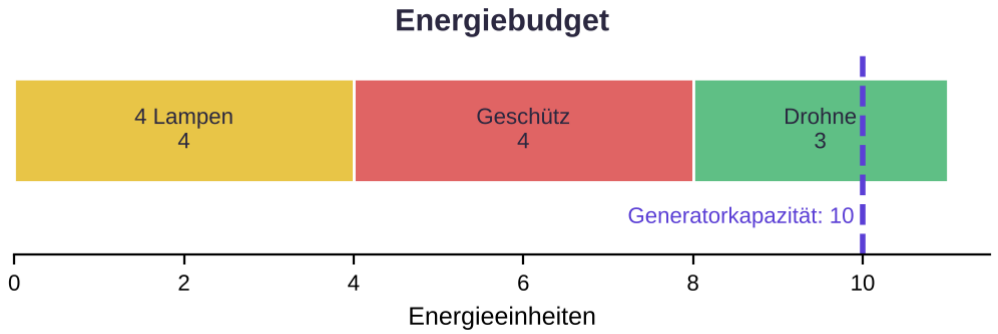
3. Core Loop

1. Die Mission beginnt mit einer Vorbereitungsphase: Die Lampen sind an, Geschütz und Drohne aus.
2. Du planst die Energieverteilung. Mit der Leertaste startest du die Welle früher.
3. Gegner kommen vom linken und rechten Kartenrand und suchen sich ein Ziel.
4. Während der Welle verteilst du die Energie per Mausklick um: die angegriffene Seite beleuchten, das Geschütz einschalten oder die Drohne für Reparaturen versorgen.
5. Besiegte Gegner hinterlassen Schrott.
6. Sind alle Gegner besiegt, beginnt die nächste Vorbereitungsphase – zerstörte Gebäude können jetzt mit Schrott wieder aufgebaut werden.
7. Nach der dritten Welle ist die Mission gewonnen und wird mit einem Rang bewertet. Wird der Generator zerstört → Blackout, die Mission ist verloren.

4. Mechaniken

4.1 Energiesystem

- Der Generator liefert 10 Energieeinheiten.
- Ein Klick auf ein Gebäude schaltet es an oder aus. Einschalten klappt nur, wenn genug freie Energie übrig ist – sonst wird die Anfrage verweigert und die Energieanzeige blinkt rot.
- Sinkt die Kapazität unter den aktuellen Verbrauch, schalten sich zuerst die zuletzt eingeschalteten Verbraucher ab, bis der Verbrauch wieder passt (Lastabwurf).



Lampen, Geschütz und Drohne brauchen zusammen 11 Einheiten – eine mehr, als der Generator liefert.

4.2 Gebäude

GEBÄUDE	ENERGIE	HP	FUNKTION
Lampe (×4)	1	4	Beleuchtet einen Kreis um sich herum. Nur Gegner im Licht können anvisiert werden.
Geschütz	4	8	Schießt einen Laser auf den nächsten beleuchteten Gegner im Umkreis von 5 Feldern, zwei Schüsse pro Sekunde.
Reparaturdrohne	3	5	Fliegt zum am stärksten beschädigten Gebäude im Umkreis von 6 Feldern und repariert es (2 HP pro Sekunde).
Generator	–	20	Liefert die Energie und gibt selbst ein schwaches Licht ab. Wird er zerstört, ist die Mission verloren.

Alle Gebäudewerte liegen in StructureData-ScriptableObjects (TurretData und RepairData erweitern sie), sodass das Balancing im Inspector passiert statt im Code.

4.3 Licht & Sichtbarkeit

- Es gibt kein Umgebungslicht: Alles außerhalb einer eingeschalteten Lampe ist komplett schwarz.
- Der sichtbare Lichtradius einer Lampe ist gleichzeitig ihr spielerischer Radius. Bevor das Geschütz zielt, prüft es, ob eine Position beleuchtet ist. Dabei zählen

nur 80 % des sichtbaren Radius, damit Gegner am schwachen Rand des Lichts nicht anvisiert werden.

- Der Generator hat ein eigenes schwaches Licht, damit die letzte Verteidigungslinie immer verteidigt werden kann.

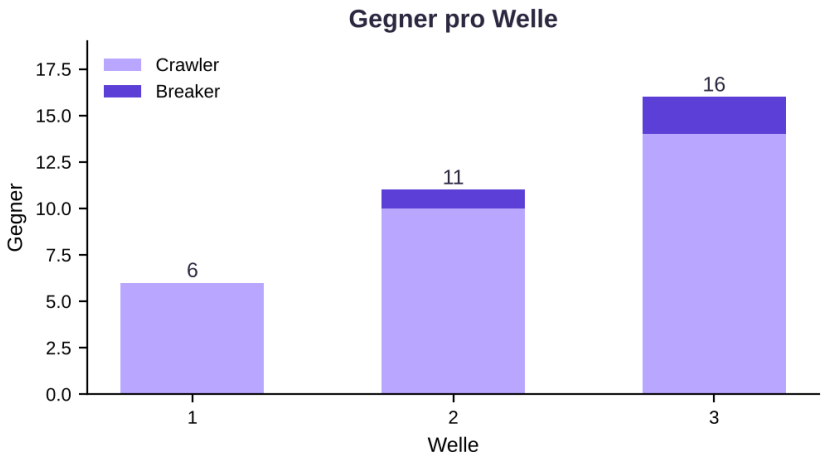
4.4 Gegner (2 Typen)

TYP	VERHALTEN	BEDROHUNG
Crawler	Klein und schnell (3 HP). Greift das nächste Gebäude an – oft eine Lampe. Kommt in Schwärmen.	Mittel
Breaker	Langsam und zäh (12 HP). Ignoriert alles andere und läuft direkt zum Generator, wo er hohen Schaden macht (3 pro Treffer).	Hoch

Beide Gegnertypen laufen über denselben EnemyController. Sie unterscheiden sich nur in ihren EnemyData- und TargetingStrategy-Assets (nächstes Gebäude gegenüber Generator zuerst, mit Ausweichstrategie, falls der Generator weg ist). Ein neuer Gegnertyp braucht also vor allem neue Daten, keinen neuen Code.

4.5 Wellen

- Drei Wellen, jeweils in einem WaveData-ScriptableObject als parallel laufende Spawn-Gruppen definiert (Gegnertyp, Anzahl, Startverzögerung, Intervall).
- 30 Sekunden Vorbereitung vor der ersten Welle, 20 Sekunden zwischen den Wellen. Mit der Leertaste lässt sich der Countdown überspringen.
- Eine Welle endet, wenn alle ihre Gegner besiegt sind.



Welle 2 bringt den ersten Breaker, Welle 3 kombiniert den größten Crawler-Schwarm mit zwei Breakern.

4.6 Schrott & Wiederaufbau

- Besiegte Gegner hinterlassen Schrott: Crawler 1, Breaker 5.
- Zerstörte Gebäude (außer dem Generator) geben ihre Energie frei und hinterlassen eine Ruine.
- In der Vorbereitungsphase zeigen Ruinen Namen und Wiederaufbaukosten – grün, wenn bezahlbar, rot, wenn nicht. Ein Klick baut das Gebäude wieder auf.
- Wiederaufgebaute Gebäude kommen mit vollen Lebenspunkten, aber ausgeschaltet zurück und müssen erst wieder mit Energie versorgt werden.

GEBÄUDE	WIEDERAUFBAU (SCHROTT)
Lampe	3
Geschütz	8
Reparaturdrohne	6

4.7 Rang & Auswertung

- Der Ergebnisbildschirm zeigt Zeit, besiegte Gegner, verlorene Gebäude und die durchschnittliche Energieauslastung während der Wellen.
- Bei einem Sieg hängt der Rang von den verlorenen Gebäuden ab: S = keins, A = bis zu 2, B = bis zu 5, C = mehr. Wiederaufgebaute Gebäude zählen trotzdem als verloren.
- Bei einer Niederlage gibt es keinen Rang.

4.8 Playtesting & Balancing

Die Werte wurden nicht einmal festgelegt und dann stehen gelassen, sondern in mehreren Runden getestet und angepasst. Drei Änderungen kamen direkt aus dem Playtesting: Ein schwaches Umgebungslicht ließ Gegner auch außerhalb der Lampen erkennen, sodass sich das Abschalten einer Lampe kaum wie ein Verlust anfühlte – das Umgebungslicht wurde komplett entfernt. Der Generator stand anfangs im Dunkeln, wodurch ein einzelner Breaker ihn ungehindert zerstören konnte – jetzt gibt er selbst ein schwaches Licht ab. Und sobald eine Lampe zerstört war, blieb ihr Bereich für den Rest der Mission dunkel, sodass Runs schon Minuten vor dem Fall des Generators praktisch verloren waren – Ruinen und Wiederaufbau mit Schrott machen Fehler jetzt korrigierbar. Die Energiekosten sind bewusst so gewählt, dass alles gleichzeitig genau eine Einheit mehr braucht, als der Generator liefert.

4.9 Audio

- **Zwei Musik-Loops** - ein ruhiger für die Vorbereitungsphase und ein treibender während der Wellen, beide in derselben Tonart und nahtlos geloopt.
- **Crossfade** - die Musik blendet weich zwischen Vorbereitung und Welle über.

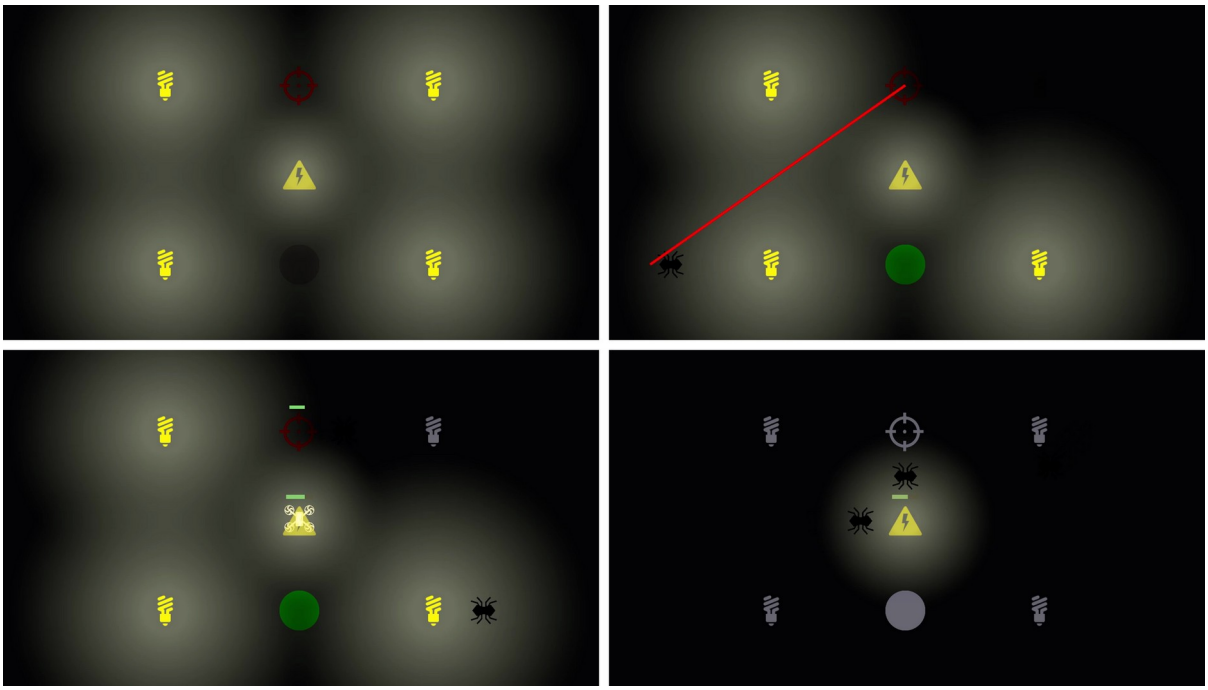
- **Stinger** - kurze Stücke für Sieg und Niederlage.
- **Ducking** - im Pausenmenü wird die Musik leiser.

5. UI / UX

- **HUD** - Energieverbrauch oben links (blinkt rot, wenn eine Anfrage verweigert wird), darunter der Schrott. Oben rechts: aktuelle Phase, Countdown, Wellennummer und verbleibende Gegner.
- **Lebensbalken** - erscheinen über einem Gebäude erst, wenn es beschädigt ist, und färben sich von grün nach rot.
- **Ruinen** - zeigen Namen und Wiederaufbaukosten, aber nur in der Vorbereitungsphase.
- **Pausenmenü** - Esc oder P: Weiter, Neustart. Eingaben auf der Karte sind währenddessen gesperrt.
- **Ergebnisbildschirm** - Sieg oder Niederlage, Rang, Statistik, Neustart-Button.
- **Steuerung** - Linksklick schaltet Energie und baut Ruinen wieder auf, Leertaste startet die nächste Welle, Esc pausiert. Keine Tastenkürzel pro Gebäude – der Blick bleibt auf der Karte, nicht auf einer Leiste.

6. Artstyle

- Alle Gebäude- und Gegnersymbole habe ich selbst gestaltet: weiße Symbole auf schwarzen Kacheln, pro Gebäude über die Sprite-Farbe eingefärbt – warmgelbe Lampen, ein rotes Geschütz, eine grüne Drohnenstation und ein violetter Generator.
- Gegner haben keine Kachel, nur dunkle Silhouetten. So verschwinden sie in der Dunkelheit und werden erst im Licht erkennbar.
- Den Look tragen die URP-2D-Lichter: warmes Lampenlicht und ein violetter Schein um den Generator. Laser, die fliegende Drohne und die Lebensbalken nutzen ein unbeleuchtetes Material, damit sie auch in völliger Dunkelheit leuchten.
- Die UI bleibt minimal, damit sie nicht mit dem konkurriert, was das Licht zeigt.

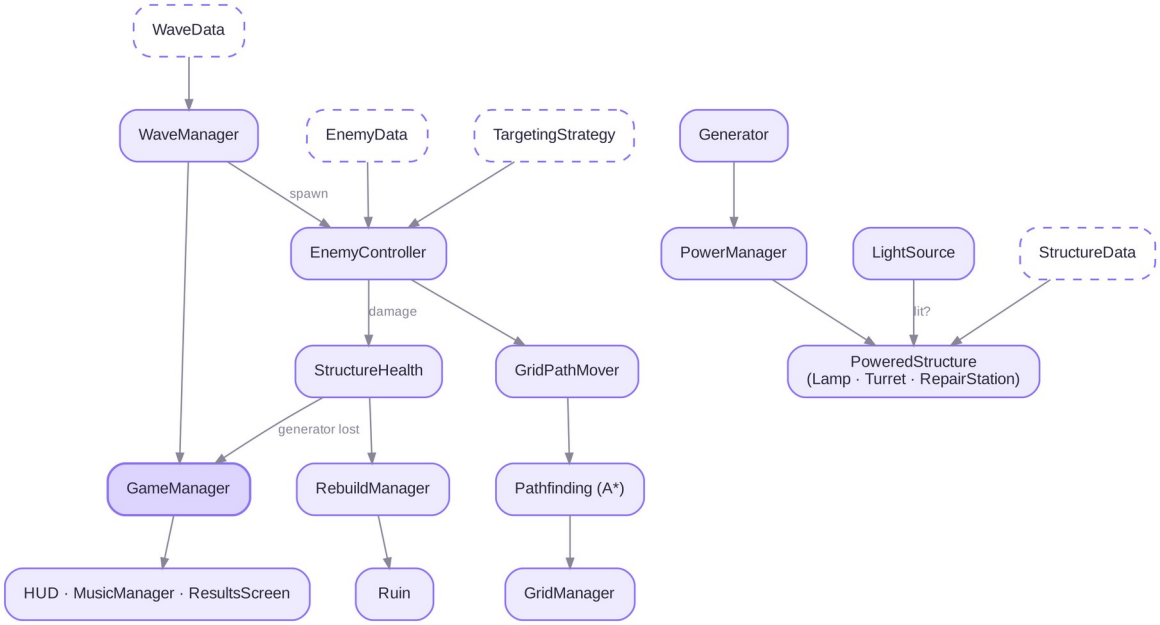


Vorbereitungsphase, Geschütz feuert auf einen beleuchteten Crawler, Drohne repariert während einer Welle, die letzte Lampe geht aus.

7. Technische Struktur

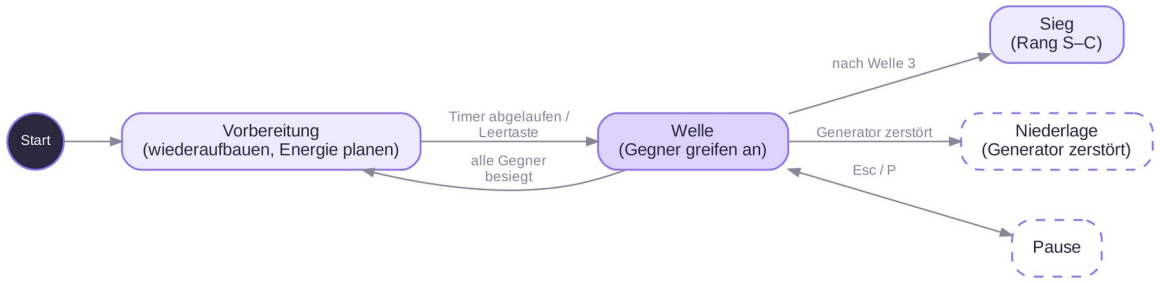
- **Grid & Pathfinding** - ein GridManager mit 20 × 12 Feldern, Tilemap-Hindernissen und Zellenbelegung. A* mit acht Richtungen, ohne Ecken zu schneiden, und Octile-Heuristik. Der GridPathMover plant neu, sobald sein nächstes Feld blockiert wird.
- **Gegner-KI** - eine Zustandsmaschine (Suchen, Laufen, Angreifen, Tot). Die Zielauswahl ist über TargetingStrategy-ScriptableObjects austauschbar.
- **Energiesystem** - PowerManager mit Kapazität, Verbrauchern und Lastabwurf. Lampen, Geschütz und Reparaturstation teilen sich die Basisklasse PoweredStructure und das Interface IPowerConsumer.
- **Licht als Abfrage** - LightSource.IsPositionLit() beantwortet, ob eine Position beleuchtet ist, mit demselben Radius wie das sichtbare Licht.
- **Datenbasiertes Design** - StructureData, TurretData, RepairData, EnemyData, WaveData und TargetingStrategy als ScriptableObjects.
- **Interfaces** - IDamageable für alles, was Schaden nimmt (Gebäude, Gegner), IPowerConsumer für alles, was Energie braucht.
- **Systeme** - GameManager für Phasen, Pause, Neustart und Statistik, dazu WaveManager, RebuildManager, MusicManager, HUD und ResultsScreen.

Systemarchitektur



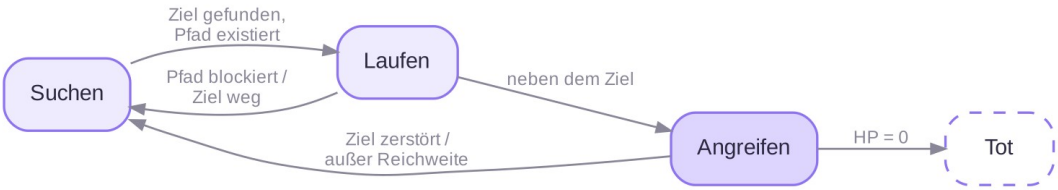
Gestrichelt = Daten-Assets. Gegner, Gebäude und Wellen bekommen ihre Werte ausschließlich über ScriptableObjects.

Spielzustände



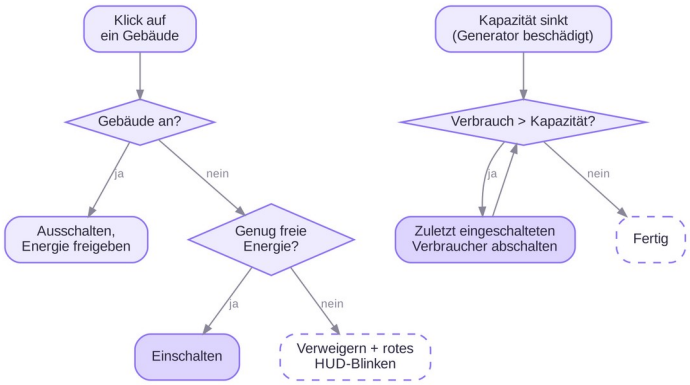
Der GameManager steuert die Phasen. Die Pause friert das Spiel über Time.timeScale = 0 ein, UI- und Musik-Übergänge laufen mit unskalierter Zeit weiter.

Zustandsmaschine der Gegner



Crawler und Breaker nutzen dieselben Zustände – nur ihre Zielstrategie unterscheidet sich.

Energiefluss & Lastabwurf



Links: ein Klick auf ein Gebäude. Rechts: was passiert, wenn die Kapazität sinkt.

8. Scope

Was tatsächlich umgesetzt ist:
1 Karte, 3 Wellen, 2 Gegnertypen, 4 Lampen, 1 Geschütz, 1 Reparaturdrohne, ein Generator, ein Energiesystem mit Lastabwurf, A*-Pathfinding, Ruinen und Wiederaufbau mit Schrott, ein Ergebnisbildschirm mit Rängen, Musik mit Überblendungen und Stingern, ein vollständiger Ablauf aus Vorbereitung, Welle, Pause und Auswertung, spielbar im Browser.

Bewusst außerhalb des Scopes:
Freies Platzieren von Gebäuden, mehrere Karten, Upgrades, weitere Gegnertypen, Progression zwischen Missionen, Soundeffekte.

Leitprinzip: Fertigstellung und Polish innerhalb des definierten Scopes haben Vorrang vor zusätzlichem Umfang.

9. Mögliche Erweiterungen

- **Schrott-Upgrades (geplant)** - die nächste geplante Erweiterung: Schrott zwischen den Wellen auch für Upgrades ausgeben, zum Beispiel größere Lichtkegel, eine schnellere Drohne, ein schneller feuerndes Geschütz oder stärkere Reparaturen. Schrott würde damit eine zweite Entscheidung erzwingen: Verlorenes wieder aufbauen oder in Stärke investieren.
- **Mehr Inhalt** - mehr Wellen, weitere Gegnertypen und zusätzliche Karten ließen sich dank des datenbasierten Aufbaus leicht ergänzen. Fürs Portfolio ist das Projekt aber bewusst klein gehalten: lieber eine Mission, die fertig und poliert ist, als eine größere, die es nicht ist.
- **Soundeffekte und Animationen** - Feedback beim Umschalten der Energie, bei Lasertreffern und zerstörten Gebäuden.